

1. Introduction to Programming

Programming is the process of creating a set of instructions that tell a computer how to perform a task.

Popular programming languages include Python, JavaScript, Java, and C++.

Programming is used in web development, app development, data science, automation, and more.

2. Variables and Data Types

Variables are containers for storing data values.

Common data types:

- Integer: Whole numbers (e.g., 10, -3)
- Float: Decimal numbers (e.g., 3.14, -0.01)
- String: Text (e.g., 'Hello, World!')
- Boolean: True or False

Example in Python:

```
name = 'Alice'
```

```
age = 25
```

```
is_student = True
```

3. Control Structures

Control structures manage the flow of a program.

- Conditional Statements: if, elif, else
- Loops: for loop, while loop

Example:

```
if age >= 18:
```

```
    print('Adult')
```

```
else:
```

```
    print('Minor')
```

```
for i in range(5):
```

```
    print(i)
```

4. Functions

Functions are reusable blocks of code that perform a specific task.

They help keep code organized and reduce repetition.

Syntax in Python:

```
def greet(name):
```

```
    print(f'Hello, {name}!')
```

```
greet('Bob')
```

5. Object-Oriented Programming (OOP)

OOP is a programming paradigm based on the concept of objects.

Key concepts:

- Class: Blueprint for creating objects
- Object: Instance of a class
- Inheritance: One class inherits properties from another
- Encapsulation: Hiding internal details

Example:

```
class Dog:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def bark(self):
```

```
        print(self.name + ' says woof!')
```

```
d = Dog('Buddy')
```

```
d.bark()
```

6. Data Structures

Common data structures include:

- List: Ordered, changeable (e.g., [1, 2, 3])
- Tuple: Ordered, unchangeable (e.g., (1, 2))
- Set: Unordered, no duplicates (e.g., {1, 2, 3})
- Dictionary: Key-value pairs (e.g., {'name': 'Alice'})

Example:

```
students = {'Alice': 90, 'Bob': 85}
```

7. Error Handling

Errors can crash programs. Handling them makes programs more robust.

Use try-except blocks:

try:

```
x = 10 / 0
```

except ZeroDivisionError:

```
    print('Cannot divide by zero!')
```

8. File Handling

Reading and writing to files is common in programming.

Example in Python:

```
with open('file.txt', 'r') as f:
```

```
    data = f.read()
```

```
with open('output.txt', 'w') as f:
```

```
    f.write('Hello!')
```